

信息安全基础综合设计实验

Lecture 03

李经纬

电子科技大学

课程回顾

Shell

➤ Shell脚本基础

- 脚本结构：`#!/bin/bash`，变量赋值`foo=bar`，访问`$foo`
- 字符串：单引号'原义'，双引号"替换"

➤ 条件判断与控制流：

- 返回码：0表示`true`，非零表示`false`
- 逻辑操作符：`&&`和`||`短路运算
- 测试表达式：`[ ]`、`[[ ]]`、`(( ))`三种形式

➤ 文本处理工具：`find`，`grep`，`awk`

数论基础实验 —— 模指数运算

模指数运算

➤目标：已知正整数 a 、 e 、 m ，计算 $a^e \bmod m$

- a ：底数； e ：指数； m ：模数
- 应用：RSA密码

➤问题：计算 $2^{90} \bmod 13$

- 提示：指数函数`pow(base, exponent)`，依赖于`math.h`；模运算符`%`

模指数运算

➤目标：已知正整数 a 、 e 、 m ，计算 $a^e \bmod m$

- a ：底数； e ：指数； m ：模数
- 应用：RSA密码

➤问题：计算 $2^{90} \bmod 13$

- 提示：指数函数`pow(base, exponent)`，依赖于`math.h`；模运算符`%`

➤如何有效计算模指数？

- 当 e 很大时， a^e **溢出**：运算结果超出固定分配空间能够存储的最大值

分治原理

➤分治法：分→治→归并

- $a^e \bmod m = a^{e_1+e_2} \bmod m = (a^{e_1} \bmod m) * (a^{e_2} \bmod m) \bmod m$

➤示例：计算 $2^{90} \bmod 13$

- 分： $2^{90} = 2^{50} * 2^{40}$
- 治： $2^{40} \bmod 13 = 3$, $2^{50} \bmod 13 = 4$
- 归并： $2^{90} \bmod 13 = 4 * 3 \bmod 13 = 12$

分治原理

➤分治法：分→治→归并

- $a^e \bmod m = a^{e_1+e_2} \bmod m = (a^{e_1} \bmod m) * (a^{e_2} \bmod m) \bmod m$

➤示例：计算 $2^{90} \bmod 13$

- 分： $2^{90} = 2^{50} * 2^{40}$
- 治： $2^{40} \bmod 13 = 3$, $2^{50} \bmod 13 = 4$
- 归并： $2^{90} \bmod 13 = 4 * 3 \bmod 13 = 12$

➤分拆规则？

二进制算法

➤以二进制方式分拆指数

- $e = d_{n-1} * 2^{n-1} + \dots + d_1 * 2 + d_0 = D_{n-1} + \dots + D_1 + D_0$
- $a^e = a^{D_{n-1}} * a^{D_{n-2}} * \dots * a^{D_1} * a^{D_0}$

➤分治计算: $A_i = a^{D_i} \bmod m = a^{2^i} \bmod m$ 或 1

- $a^{2^{i+1}} \bmod m = (a^{2^i} \bmod m) * (a^{2^i} \bmod m) \bmod m$
- 计算 $O(\log_2 e)$ 次

➤归并: $(A_{n-1} * \dots * A_0) \bmod m$

模指数运算示例 I : $7^{256} \bmod 13$

➤ **分拆指数** : $256 = 2^8$

➤ **分治计算** :

- $7^1 \bmod 13 = 7$
- $7^2 \bmod 13 = [(7^1 \bmod 13) * (7^1 \bmod 13)] \bmod 13 = 10$
- $7^4 \bmod 13 = 9$, $7^8 \bmod 13 = 3$, ...
- $7^{256} \bmod 13 = [(7^{128} \bmod 13) * (7^{128} \bmod 13)] \bmod 13 = 9$

➤ **归并** : $7^{256} \bmod 13 = 9$

模指数运算示例 II : $5^{117} \bmod 19$

➤ **分拆指数** : $117 = 1 + 4 + 16 + 32 + 64$

➤ **分治计算** :

- $5^1 \bmod 19 = 5$
- $5^2 \bmod 19 = [(5^1 \bmod 19) * (5^1 \bmod 19)] \bmod 19 = 6$
- $5^4 \bmod 19 = 17$; ... $5^{16} \bmod 19 = 16$; ... $5^{32} \bmod 19 = 9$
- $5^{64} \bmod 19 = [(5^{32} \bmod 19) * (5^{32} \bmod 19)] \bmod 19 = 5$

➤ **归并** : $5^{117} \bmod 19 = 5^{1+4+16+32+64} \bmod 19 = 1$

数论基础实验 —— 素性测试

素数

- 如果 n ($n > 1$) 是素数，当且仅当 n 只有因子1和它本身
 - 注：1不是素数
- 数学应用：任意自然数可由全体素数基底表达
- 素性测试：**判定 n 是否为素数**

素性判定基本方法

- 方法 I：判断 n 是否被 i 整除 ($i = 2, 3, \dots, n-1$)
 - 素数 n 只有1和 n 两个因子

素性判定基本方法

➤方法 I：判断 n 是否被 i 整除 ($i = 2, 3, \dots, n-1$)

- 素数 n 只有1和 n 两个因子
- $n-2$ 个判断条件，**如何减少判断次数？**

素性判定基本方法

➤方法 I：判断 n 是否被 i 整除 ($i = 2, 3, \dots, n-1$)

- 素数 n 只有1和 n 两个因子
- $n-2$ 个判断条件，**如何减少判断次数？**

➤方法II：判断 n 是否被 i 整除 ($i = 2, 3, \dots, n^{1/2}$)

- 如果 $j > n^{1/2}$ 是 n 的因子，总能找到 $i < n^{1/2}$ ，满足 $n = i * j$
- 判断次数减少为 $n^{1/2} - 1$ 次，复杂度由 $O(n)$ 降低为 $O(n^{1/2})$

素性判定基本方法

➤方法 I：判断 n 是否被 i 整除 ($i = 2, 3, \dots, n-1$)

- 素数 n 只有1和 n 两个因子
- $n-2$ 个判断条件，**如何减少判断次数？**

➤方法II：判断 n 是否被 i 整除 ($i = 2, 3, \dots, n^{1/2}$)

- 如果 $j > n^{1/2}$ 是 n 的因子，总能找到 $i < n^{1/2}$ ，满足 $n = i * j$
- 判断次数减少为 $n^{1/2} - 1$ 次，复杂度由 $O(n)$ 降低为 $O(n^{1/2})$
- **如何进一步减少判断次数？**

一种确定性素性判定思想

- 确定性：判断 n 一定是/不是素数
- 思想：找出 $\{2, 3, \dots, n^{1/2}\}$ 中的素数，判断 n 是否含有这些素因子
 - $\{2, 3, \dots, n^{1/2}\}$ 可能存在合数，合数含有素因子导致重复判断
- 步骤：（1）确定待筛选集合 $\{2, 3, \dots, n^{1/2}\}$ ；（2）基于 $\{2, 3, \dots, n^{1/2}\}$ 的**Eratosthenes筛选**；（3）筛选后元素与 n 整除判定

Eratosthenes筛选法

- 目标：**范围N内的素数**
 - 不适用于计算某个范围内全部素数
- 取**第一个素数**2, 划去 $\{2, \dots, N\}$ 中除2以外所有2的倍数
- 大于2的第一个正整数(即3)被认定为素数, 在余下的整数中划去除3以外所有3的倍数
- 循环此过程直到找到 $\{2, 3, \dots, N\}$ 中的所有素数

Eratosthenes筛选法示例

	2	3	4	5	6	7	8	9	10	Prime numbers
11	12	13	14	15	16	17	18	19	20	
21	22	23	24	25	26	27	28	29	30	
31	32	33	34	35	36	37	38	39	40	
41	42	43	44	45	46	47	48	49	50	
51	52	53	54	55	56	57	58	59	60	
61	62	63	64	65	66	67	68	69	70	
71	72	73	74	75	76	77	78	79	80	
81	82	83	84	85	86	87	88	89	90	
91	92	93	94	95	96	97	98	99	100	
101	102	103	104	105	106	107	108	109	110	
111	112	113	114	115	116	117	118	119	120	

基于筛选法素性测试

判断2543是否为素数

- **求平方根** : $2543^{1/2} \sim 50$, 确定待筛选集合 $\{2, 3, 4, \dots, 50\}$
- 使用**Eratosthenes筛选法**在集合 $\{2, 3, 4, \dots, 50\}$ 中筛选出所有素数: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47
- **整除判定** : 2543除以这15个素数 , 结果都不是整数 , 因此2543**一定是素数**

版本控制工具

版本控制

➤版本控制系统 (Version Control System, VCS)

- 记录文件内容变化，以便将来查阅特定版本修订情况的系统
- 跟踪目录和文件的修改历史

➤解决的问题：

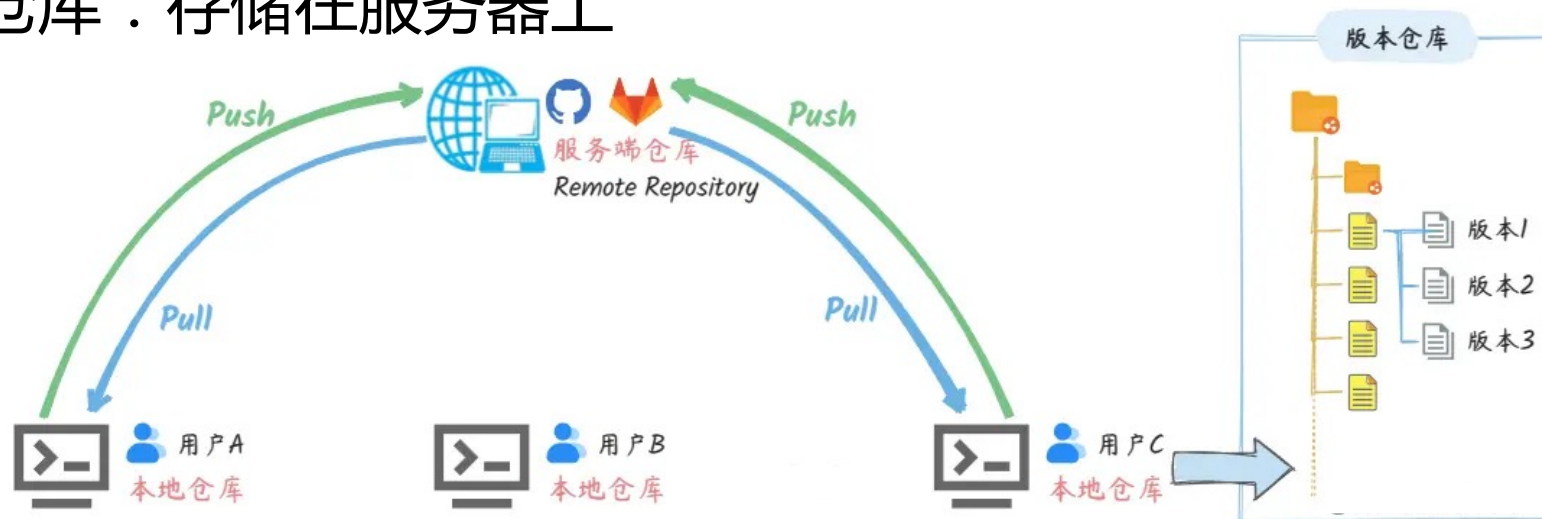
- 避免"final.doc, final_v2.doc, final_final.doc"的混乱
- 多人协作时的文件冲突和覆盖问题
- 代码回退：当新功能出现bug时快速回到稳定版本
- 实验性开发：在不影响主线代码的情况下尝试新功能

基本概念 I

➤ 关注分布式版本控制系统git

➤ 仓库 (Repository)

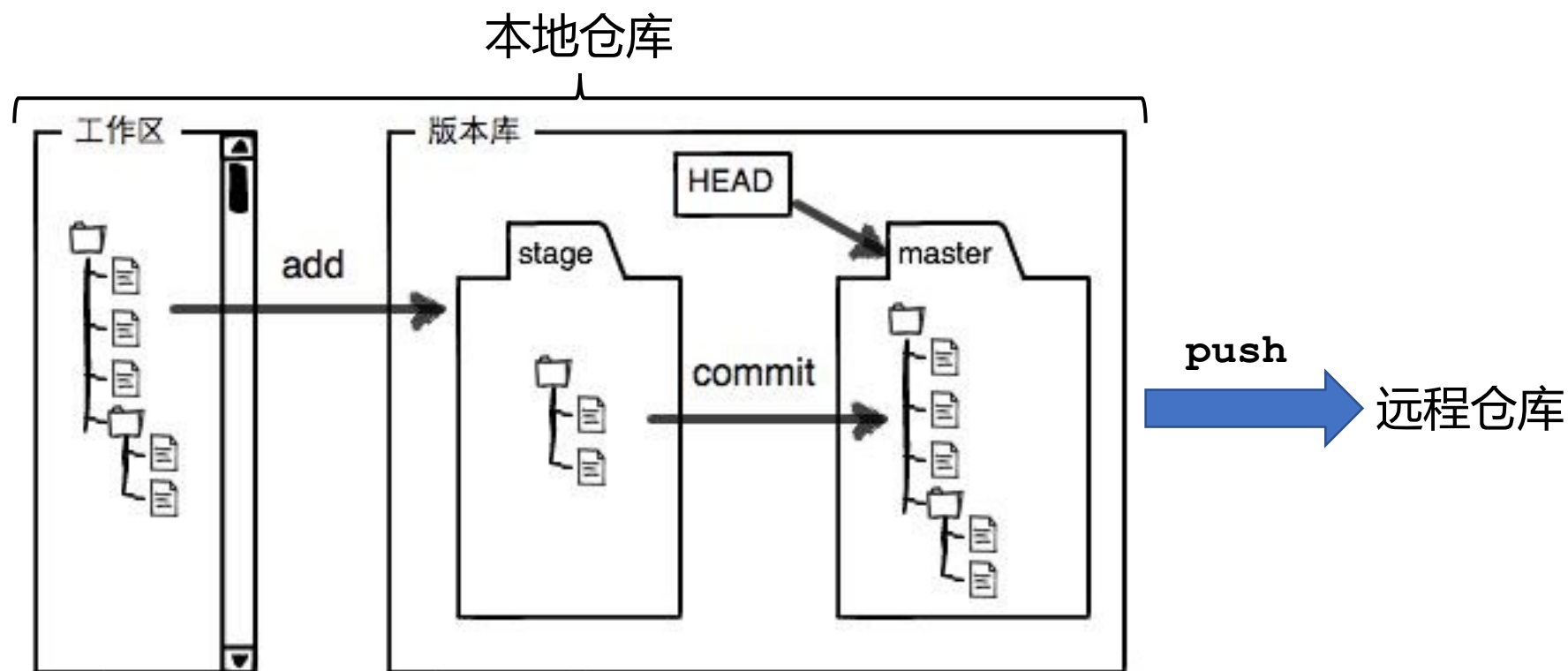
- 本地仓库：存储在本地的完整历史
- 远程仓库：存储在服务器上



基本概念 II

➤提交（Commit）：某个时间点本地工作区的快照

- 快照永久性存储到 `.git/` 目录



Git安装与配置

➤安装Git

- Linux : `sudo apt install git` (Ubuntu/Debian)
- macOS : `brew install git`
- Windows : 下载Git for Windows

➤首次配置

```
git config --global user.name "Your Name"  
git config --global user.email "Your.email@example.com"
```

本地初始化

➤本地创建新目录并初始化

```
cd my-project  
git init
```

初始化后的结构目录

my-project/

└─ .git/

│ config

│ objects/

│ refs/

└─ HEAD

Git仓库的核心目录

项目特有的配置选项

存储所有数据内容

存储指向数据的提交对象的指针

指向目前被检出的分支

记录更新

➤检查当前文件状态

- 不必要但是建议查看当前状态：`git status`

➤将新/更新文件放入暂存区

```
git add .           # 添加当前目录所有文件
git add README.txt  # 添加单个文件
```

➤提交暂存区内容

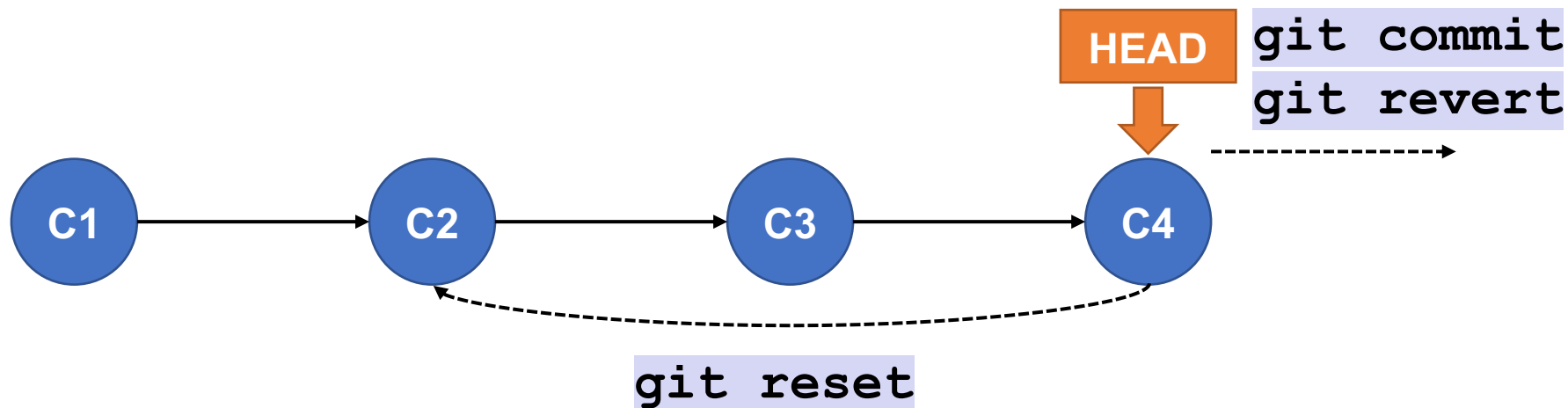
```
git commit -m "提交备注信息" # 提交暂存区内容
```

查看提交历史

➤ 基本历史查看

<code>git log</code>	# 详细提交历史
<code>git log --oneline</code>	# 每个提交一行显示

➤ 版本控制：在若干提交形成的有向图中移动HEAD指针



撤消操作：reset

➤重置到指定提交

重置到上一个提交，新提交的所有更改进入暂存区，工作目录仍存有这些文件

```
git reset --soft HEAD~1
```

重置到上一个提交，新提交的所有更改被彻底删除，暂存区和工作目录回到上一个提交的状态

```
git reset --hard HEAD~1
```

重置到上一个提交，工作目录仍存有新提交的文件，但未放入暂存区

```
git reset [--mixed] HEAD~1
```

- ~ 符号沿着当前分支的提交历史，回溯父提交链

撤销操作：revert

➤安全撤销指定提交

- 新创建一个提交，用于撤销指定提交的修改

```
# 创建一个新提交，逆向执行{commit-hash}的更新  
git revert {commit-hash}
```

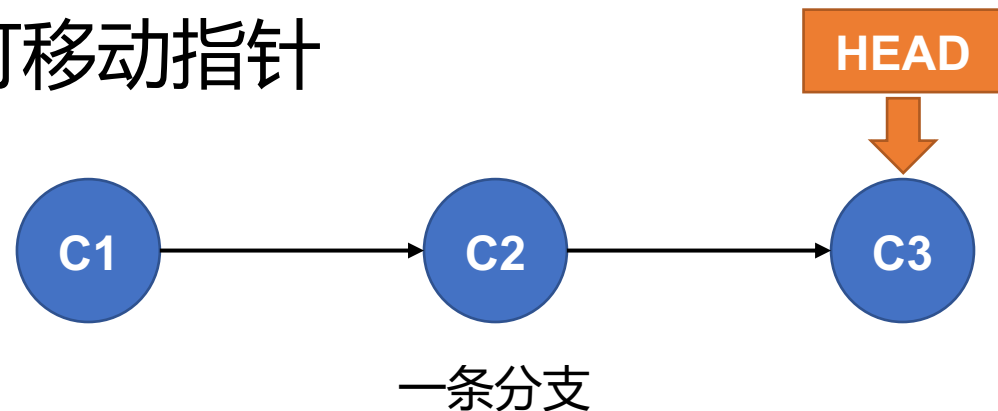
特性	git revert	git reset
工作方式	创建一个新的提交来撤销更改	移动 HEAD 指针，可以删除或回退提交
提交历史	保持不变，提交历史是线性的	会改写历史记录，某些提交可能被删除
安全性	安全，不会改写历史记录	危险，使用 git reset --hard会永久删除提交

Git分支操作

分支简介

➤ Git分支本质上是指向提交对象的可移动指针

- HEAD指向**当前**所在的本地分支
- 默认分支名字是main或master



➤ 为什么需要分支

- 实验性开发：在分支上试验新功能，不影响主线
- 发布管理：维护不同版本的代码
- 并行开发：不同功能可以在不同分支上同时进行，方便多人协作

分支创建与切换

➤查看分支

```
git branch # 查看所有本地分支
```

➤创建并切换分支

```
git branch testing # 创建名为testing的新分支  
git checkout testing # 切换到testing分支
```

➤分支切换注意事项

- 切换分支前，**确保当前分支的更改已提交或暂存**
- 使用`git stash`可以临时保存未提交的更改
 - 注：`git stash`是储藏栈，用于临时保存未提交的更改，能够回到干净的状态

附：git stash常用指令

命令	描述
<code>git stash push</code>	暂存当前所有未提交的更改（包括已暂存和未暂存的）
<code>git stash list</code>	查看储藏栈中所有储藏列表，例如唯一标识符stash@{0}
<code>git stash apply</code>	将最近的储藏应用到当前工作目录，但保留这个储藏在栈中
<code>git stash pop</code>	将最近的储藏应用到当前工作目录，并删除这个储藏
<code>git stash apply <stash-id></code>	应用指定的储藏，例如 <code>git stash apply stash@{1}</code>
<code>git stash drop <stash-id></code>	删除指定的储藏，例如 <code>git stash drop stash@{0}</code>
<code>git stash clear</code>	清空储藏栈，删除所有储藏
<code>git stash show</code>	查看最近一个储藏的详细更改

分支合并：快进合并

```
git checkout main          # 切换到主分支
git merge hotfix           # 将hotfix分支合并到main
```

➤条件：

- 合并目标分支是在当前分支基础上创建
- 当前分支没有新的提交

```
# 合并前
main      A---B---C
           \ 基于main创建了hotfix分支
hotfix    D---E

# 合并后
main      A---B---C---D---E
```

分支合并：三方合并

- 条件：两个要合并的分支都有各自独立的提交历史
 - 例如在开发hotfix分支的同时，main分支有新的提交

```
# 合并前
main      A---B---C---F---G
基于main创建了hotfix分支 \
hotfix      D---E---      /
```

- 使用两个分支的末端提交（G和E）以及共同祖先（B）进行合并，
创建一个新的合并提交
 - 当合并产生冲突时，需要手动解决

分支管理

➤查看分支详细信息

<code>git branch -v</code>	# 查看每个分支的最后一次提交
<code>git branch --merged</code>	# 查看已合并到当前分支的分支
<code>git branch --no-merged</code>	# 查看未合并的分支

➤删除分支

<code>git branch -d feature</code>	# 删除已合并的分支
<code>git branch -D feature</code>	# 强制删除分支（未合并也删除）

Git远程仓库

远程仓库

➤ 托管在远程站点（例如Github）或其他地方的仓库

➤ 添加远程仓库

```
# 本地初始化后
# user 是github用户名, repo是仓库名
git remote add origin https://github.com/user/repo.git

# 列出远程仓库的简短名称
git remote
```

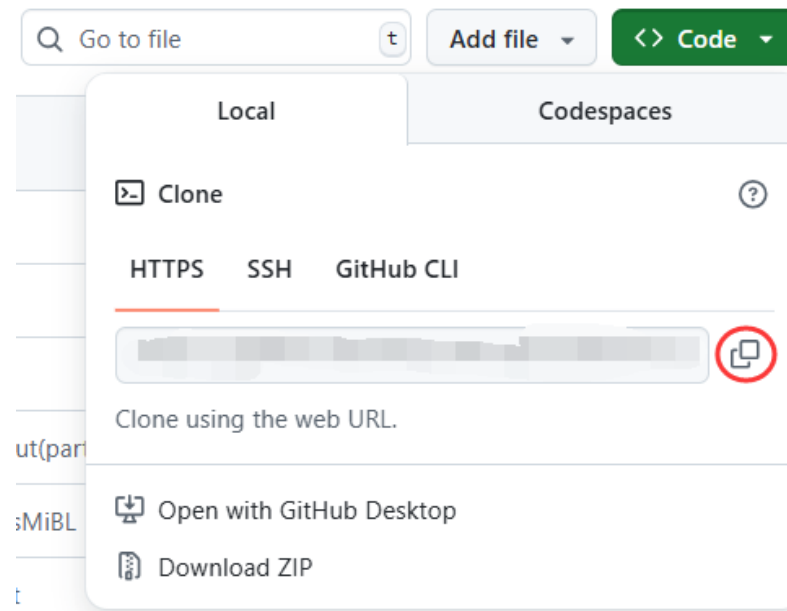

远程初始化

➤新建远程仓库

- 在github的repositories界面点击New并创建一个repo

➤克隆到本地：

- 点击图中按钮复制url
- 终端中输入`git clone [ctrl v]`



远程仓库管理

➤从远程仓库获取提交和数据

```
# 从远程仓库下载最新提交和数据，并与当前本地分支合并  
# origin为远程仓库默认名称，main为远程仓库默认分支  
git pull [origin main]
```

➤推送本地提交和数据到远程仓库

```
# 将本地提交和数据同步到远程仓库  
git push [origin main]  
# 将本地分支推送到远程仓库，并设置本地分支与远程origin/feature分支关联  
git push -u origin feature
```

自学资源

- Pro Git : <https://git-scm.com/book/zh/v2> , 在线书的内容是如何使用Git以及Git内部原理。
- Learn Git Branching: <https://learngitbranching.js.org> , 通过基于浏览器的游戏来学习 Git

课堂作业

作业要求

➤作业内容：模指数运算+埃氏素性测试

➤评分等级：

- **课堂上**完成作业，正确解释代码，通过助教登记，获得平时成绩加分
- 课后按时（本周日24点之前）完成作业并发送至邮箱**zihasyu@proton.me**，获得本次作业平时成绩
 - 邮件题目：【周X】姓名+学号
- 未完成作业，无法获得本次作业平时成绩